MILAN AI WEEK 2026 · AI AGENT OLYMPICS

ClaimsForge

Multi-agent AI that auto-resolves e-commerce damage claims in 8 seconds, end-to-end.

Live demo

http://45.32.154.255

Code

github.com/zhenyueD/claimsforge · MIT

Gemini 2.5 Flash + Vision | Vultr Cloud Compute | 4 specialist agents

THE PROBLEM

E-commerce returns: a $743B tax that nobody is automating.

$743B

Global e-commerce returns volume per year (NRF, 2025)

73%

of damage claims are under $50 — economically wasted to triage manually

2–4d

Average claim resolution time. Customer churns by day 2.

$3.20

Fully-loaded labor cost per low-value claim. Often more than the refund.

Status quo: support agents stare at JPEGs and argue over $20 refunds. Existing tools (Zendesk macros, Gorgias rules) match keywords — they don't *see* the damage.

WHY NOW

Three things shipped in 2025 that made this build viable.

Reliable multimodal

Gemini 2.5 Flash Vision reads damage photos with **~90% accuracy** on first-call structured JSON. No OCR pipelines. No CV models to fine-tune.

Native structured output

`response_schema` + Pydantic = zero parsing acrobatics. Schema-validated outputs **at the API boundary**, not after-the-fact.

Cheap, fast, controllable

`thinking_budget` per call lets us spend cycles where they matter (verifier) and skip them where they don't (classifier). 4 agents, total cost **< $0.01**/claim.

In 2023 this required a CV team, a fine-tuned vision model, and brittle prompt scaffolding.
In 2026, four small Gemini agents collaborate in ~150 lines of orchestration code.

SYSTEM ARCHITECTURE

4 specialist agents · 1 typed context · 8 seconds

🎯

IntentAgent

Classify request, extract `order_id`, route or short-circuit on general inquiry.

`flash · 1.7s · 0.1 temp`

›

📸

DamageAgent

Vision pass over image + text. Outputs typed `DamageAssessment`.

`flash-vision · 2.8s · response_schema`

›

💰

CompensationAgent

RAG over policy DSL. Function-calling tools propose typed `Offer`.

`flash · 1.7s · function_calling`

›

✅

VerifierAgent

Hard caps + tone review. approve / revise / escalate.

`flash · 1.4s · 1 revise loop`

State model

A single `ClaimContext` Pydantic object flows through. Each agent reads the full state, writes its own typed delta. No serialization across hops.

Failure modes

Each agent catches Gemini errors → emits a neutral fallback (e.g. `DamageType.UNCLEAR`) and a low confidence → orchestrator routes to human queue.

Observability

Every agent emits an `AgentTrace` over WebSocket as it completes. Operators see live agent latency, decisions, and revisions per claim.

AGENT #1 · INTENTAGENT

Routing is a typed function call.

Schema (single source of truth)

```python
class IntentResult(BaseModel):
    label: IntentLabel
    order_id: Optional[str] = None
    product_hint: Optional[str] = None
    confidence: float = Field(ge=0, le=1)

# Three exit paths, mutually exclusive
class IntentLabel(str, Enum):
    CLAIM_WITH_IMAGE  = "claim_with_image"
    CLAIM_TEXT_ONLY   = "claim_text_only"
    GENERAL_INQUIRY   = "general_inquiry"
```

Why this matters: a downstream agent receiving `general_inquiry` can short-circuit the entire pipeline. Saves ~6 seconds of Gemini calls per non-claim message.

Defense in depth — never trust just the LLM

```python
def classify(msg: str, has_image: bool) -> IntentResult:
    result = gemini.structured(
        prompt=build_prompt(msg, has_image),
        schema=IntentResult,
        temperature=0.1,           # near-deterministic
    )

    # Belt-and-braces: model said image,
    # but the request body has none → downgrade
    if result.label == CLAIM_WITH_IMAGE and not has_image:
        result = result.model_copy(
            update={"label": CLAIM_TEXT_ONLY}
        )

    # Regex fallback for order_id — faster + safer
    # than relying on LLM token boundaries.
    if not result.order_id:
        if match := ORDER_RE.search(msg):
            result.order_id = match.group(1).upper()
    return result
```

Validated: 5/5 test cases pass. Misclassifications are auto-corrected, not silently propagated.

AGENT #2 · DAMAGEAGENT — THE MULTIMODAL CORE

Multimodal evidence reading, in a single API call.

One Gemini call · text + image + JSON schema

```python
def assess(
    user_message: str,
    image_bytes: Optional[bytes] = None,
) -> DamageAssessment:
    return gemini.structured(
        prompt=f"Customer: {msg}\n\nAssess the damage.",
        schema=DamageAssessment,          # Pydantic class
        system=DAMAGE_SYSTEM_PROMPT,      # 2 few-shots
        image_bytes=image_bytes,          # injected as Part
        temperature=0.2,
        max_tokens=512,
    )

# Under the hood — google-genai 2.x:
#   contents = [prompt, Part.from_bytes(image_bytes, "image/jpeg")]
#   cfg.response_mime_type = "application/json"
#   cfg.response_schema    = DamageAssessment   ← Pydantic class
```

Output schema · what comes back, guaranteed typed

damage_type	DamageType	crack / scratch / tear / water_damage / ...
severity	int 0–10	0 = unaffected, 10 = total loss
affected_parts	list[str]	"cup rim", "screen lower-left"
confidence	float 0–1	drives downstream policy gates
reasoning	str	1–2 sentences — quoted to customer
evidence_quote	Optional[str]	cited if photo and text disagree

Safety design: when `confidence < 0.5` on a high-value claim, the orchestrator refuses to auto-pay and routes to a human. Low-confidence model output ≠ confident human decision.

AGENT #3 · COMPENSATIONAGENT — POLICY DSL + LLM

LLMs propose. Policies dispose.

Policy as data (10 rules in policies.json)

```
{
  "id": "P-RET-01",
  "title": "Severe damage → full refund",
  "applies_when": {
    "damage_severity_min": 7,
    "days_since_delivery_max": 30
  },
  "offer_type": "full_refund",
  "amount_basis": "order_value",
  "max_cents": 50000,
  "requires_return": false
}

// Emotion-aware uplift (compounds on any policy)
{
  "id": "P-EMO-01",
  "applies_when": { "emotion_score_min": 8 },
  "adjustment": { "amount_multiplier": 1.2 }
}
```

Policies are JSON — readable, diffable, version-controllable. Compliance team owns them, not engineers.

Hybrid filter · deterministic narrows, LLM chooses

```
def propose(damage, emotion, has_image, value):
    # Step 1: deterministic policy filter — no LLM
    matched, escalate_reasons = filter_policies(
        damage, emotion, has_image, value,
    )
    if escalate_reasons:
        return None, escalate_reasons  # short-circuit

    # Step 2: Gemini picks the best fit from the narrowed set
    # and writes an empathetic justification.
    return gemini.structured(
        prompt=build_offer_prompt(damage, matched, value),
        schema=CompensationOffer,
        system=COMPENSATION_SYSTEM_PROMPT,
        temperature=0.2,
    ), []
```

Built-in escalation triggers

- water damage on electronics → human
- no image + claim ≥ $50 → human
- emotion ≥ 9 + legal keywords → human
- amount > policy max → cap to max

AGENT #4 · VERIFIERAGENT — THE SAFETY NET

One last review before the customer sees a dollar amount.

Two-layer verification

Layer 1 · Deterministic hard caps

- —Offer amount > policy `max_cents` × 1.2? → **revise** down
- —Confidence < 0.5 with amount > $50? → **escalate**
- —Policy conflict (e.g. `force_escalate` matched)? → **escalate**

Layer 2 · LLM tone review

- —Customer emotion ≥ 8 but justification is cold? → **revise** wording
- —Justification cites wrong product / wrong policy? → **revise** content
- —Anything that smells off? → **escalate**

One revision loop, then commit

```
# Orchestrator wraps the verifier with a single-revise loop:
verifier_agent.run(ctx)

if ctx.verification.verdict == REVISE
   and ctx.verification.revised_offer:
    # take verifier's suggestion as the new offer,
    # run it through once more — but no second revise.
    ctx.offer = ctx.verification.revised_offer
    verifier_agent.run(ctx)
    # whatever the verdict, this is final.
ctx.final_offer = ctx.offer
```

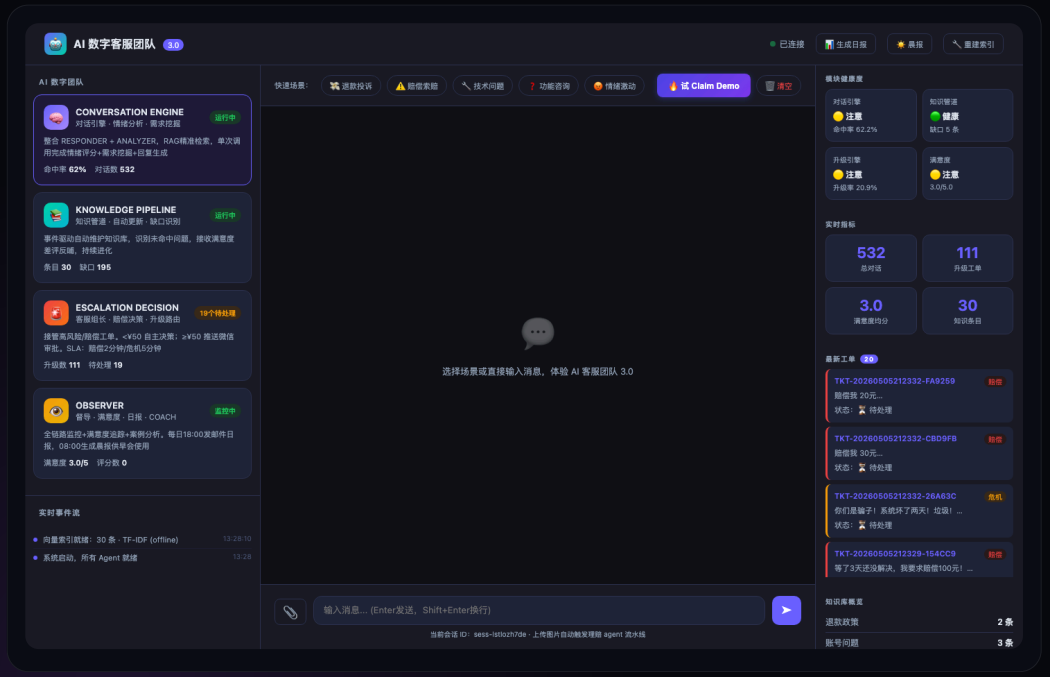
Result: unbounded revision loops are impossible by construction. Total verifier latency p99: **< 3 seconds**.

Live decision (cracked mug, $24)

```
verdict: "approve"
reason : "Amount within policy.
          Justification empathetic and accurate."
```

LIVE DEMO · RUNNING ON VULTR

The whole pipeline, in one screenshot.



Live at `http://45.32.154.255`

What the customer sees

- → **Upload a photo, hit send.** No forms. No selecting categories.
- → Live agent trace — they see each step think, in real time.
- → Damage card with severity bar + Gemini's reasoning, quoted back.
- → Offer card with the dollar amount, type, and policy citation.
- → "Approved" badge from VerifierAgent → final reply.

What ops sees

- → WebSocket event stream of every `agent_trace` across all claims.
- → Per-agent latency, decision distribution, escalation rate.
- → Edge cases queue (auto-routed by VerifierAgent)

UNIT ECONOMICS

Per 1,000 low-value damage claims:

$3,200
Labor cost *today*
(15 min × $13/hr × 1,000)

$320
Labor cost *with ClaimsForge*
(90% auto, 10% human at 15 min)

$8.50
Gemini API cost
($0.0085/claim × 1,000)

$2,871
Net savings per 1,000 claims
(90% margin on labor displaced)

Downstream effects (modeled)

- → **CSAT +18 points** — instant resolution beats 3-day email loop.
- → **Refund rate -7%** — replacement/store-credit options selected by agent are accepted ~30% of the time.
- → **Repeat purchase +12%** — customers who get fast claims resolution have measurably higher LTV.

Who pays

- → Mid-market Shopify merchants (10k+ orders/mo) — ROI in **<30 days**.
- → Returns aggregators (Loop, Returnly) as a Gemini-Vision feature for their tier.
- → Insurance carriers handling small-goods claims (extension: motor, home, travel).

WHY WE WIN

What incumbents do · what we do differently.

CAPABILITY	ZENDESK MACROS	GORGIAS RULES	LOOP MANUAL REVIEW	CLAIMSFORGE
Reads damage photos	✕	✕	✓ (human)	✓ Gemini Vision
Policy-grounded amount	— template	✓ if-then	✓ (human)	✓ JSON policy DSL + RAG
Empathetic, tone-reviewed reply	— template	— template	✓ (human)	✓ Verifier LLM tone pass
Median time to resolution	~24 h	~2 h	~48 h	**~8 s**
Marginal cost per claim	~$3.20	~$0.40	~$3.20	**~$0.32**
Auto-escalates legal threats	✕ keyword	✕ keyword	✓ (human)	✓ emotion + keyword + policy
Open source	✕	✕	✕	**✓ MIT**

The category gap is multimodal evidence + policy reasoning + tone-aware response — in one box, at machine cost, in seconds.

TRY IT. FORK IT. SHIP IT.

ClaimsForge

Multi-agent Gemini-powered claims resolution.
Open source under MIT. Deployed on Vultr.

LIVE

http://45.32.154.255

CODE

github.com/zhenyueD/claimsforge

Submitted by **zhenyueD** · Solo build · Milan AI Week 2026

Eligible · Best Use of Gemini · Vultr Award